

~/LEAFLENS

LeafLens

*AI-powered houseplant identification and
care assistant.*

Luigy Leonardo

Artificial Intelligence (BEINSOF52)

Instructor: Juan Antonio Castro Silva

May 2026

Table of contents

 LeafLens	•
 What it does	•
 Headline results	•
 Architecture	•
 Quick start	•
 Tech stack	•
 Project structure	•
 Documentation	•
 Cleanup	•
 License	•
1. Introduction	•
Background	•
Project overview	•
Scope	•
Document structure	•
2. Problem Statement	•
The pain point	•
Secondary problem: pet safety	•
What is needed	•
Why this is a good fit for artificial intelligence	•
Out of scope	•
3. Objectives	•
General objective	•
Specific objectives	•
Success criteria	•
4. State of the Art	•
4.1 Plant identification as a computer vision problem	•

4.2 Public datasets	•
4.3 Commercial products	•
4.4 Approaches relevant to LeafLens	•
4.5 Gap LeafLens addresses	•
References	•
5. Requirements	•
5.1 Functional requirements	•
5.2 Non-functional requirements	•
5.3 Constraints	•
5.4 Assumptions	•
6. Use Cases & User Stories	•
6.1 Actors	•
6.2 Use case diagram (textual)	•
6.3 Use case details	•
6.4 User stories	•
7. Data Dictionary & Entity-Relationship Model	•
7.1 Logical entities	•
7.2 Entity-relationship diagram (textual)	•
7.3 Data dictionary	•
8. Class Diagrams	•
8.1 Model layer	•
8.2 Service layer (Flask + SQLAlchemy)	•
8.3 Service layer responsibilities	•
8.4 Frontend component tree (React)	•
9. GUI Mockups	•
9.1 Home page (desktop)	•
9.2 Home page (mobile)	•
9.3 Loading state	•
9.4 Result card (high confidence)	•

9.5 Result card (low confidence)	•
9.6 Catalog grid	•
9.7 About page	•
10. API Catalog	•
10.1 Endpoint summary	•
10.2 GET /api/health	•
10.3 POST /api/predict	•
10.4 GET /api/species	•
10.5 GET /api/species/{id}	•
10.6 Rate limits and quotas	•
11. Testing	•
11.1 Strategy summary	•
11.2 Unit tests	•
11.3 Functional tests	•
11.4 Integration tests	•
11.5 Continuous integration	•
11.6 Manual testing checklist	•
12. Architecture	•
12.1 High-level architecture	•
12.2 Component responsibilities	•
12.3 Inference data flow (sequence)	•
12.4 Deployment topology	•
12.5 Configuration & secrets	•
12.6 Scalability notes	•
13. Results and Discussion	•
13.1 Experimental setup	•
13.2 Architectures compared	•
13.3 Headline metrics	•
13.4 Hyperparameters and search space	•

13.5 Cross-validation strategy	•
13.6 Qualitative results	•
13.7 End-to-end latency	•
13.8 Discussion	•
13.9 Limitations	•
14. Recommendations and Future Work	•
14.1 Recommendations	•
14.2 Future work	•
14.3 Lessons learned	•
15. Presentation Outline (English)	•
Slide 1 — Title (15 s)	•
Slide 2 — The problem (45 s)	•
Slide 3 — What LeafLens does (45 s)	•
Slide 4 — Live demo (90 s)	•
Slide 5 — Dataset (30 s)	•
Slide 6 — Architectures compared (60 s)	•
Slide 7 — Results (60 s)	•
Slide 8 — Where the model still fails (45 s)	•
Slide 9 — System architecture (60 s)	•
Slide 10 — Documentation and engineering practices (45 s)	•
Slide 11 — Future work (30 s)	•
Slide 12 — Thanks + Q&A (15 s)	•
Speaker notes — common questions to prepare for	•
16. Deployment Guide	•
0. Prerequisites	•
1. Backend — Render	•
2. Frontend — Vercel	•
3. Smoke test the full deployment	•
4. Render free-tier notes	•
5. Rollback	•

6. Tear-down (when the project is done) •

17. Presentation Cheat Sheet — Numbers, Paths, and Q&A Reference •

 The headline numbers •

 Local paths (your laptop) •

 URLs •

 Architecture details (in case they grill you) •

 Training recipe (identical for all 3 architectures) •

 Care card data sources •

 Likely questions and crisp answers •

 Demo flow (for the live demo slide) •

LeafLens — Folder Inventory •

Top-level map •

Where each thing is •

Running the project from this folder •

Reproducing the training results •

Quick checks (defense backup) •

Cleanup (after grading) •



AI-powered houseplant identification and care assistant. Upload a photo, get the species, get the care card, know if it's safe for your pets.

✨ What it does

Take a photo of any houseplant. LeafLens identifies the species among **47 of the most common indoor plants worldwide**, and returns:

- 🧠 Top-1 species name + confidence + 2 alternatives
- 💧 Watering schedule and light requirements
- 🌡️ Temperature, humidity, and fertilizer recommendations
- 🐾 Pet safety badge (toxic / mild / safe for cats and dogs, sourced from ASPCA)

End-to-end latency under **1 second** on the deployed instance.

📊 Headline results

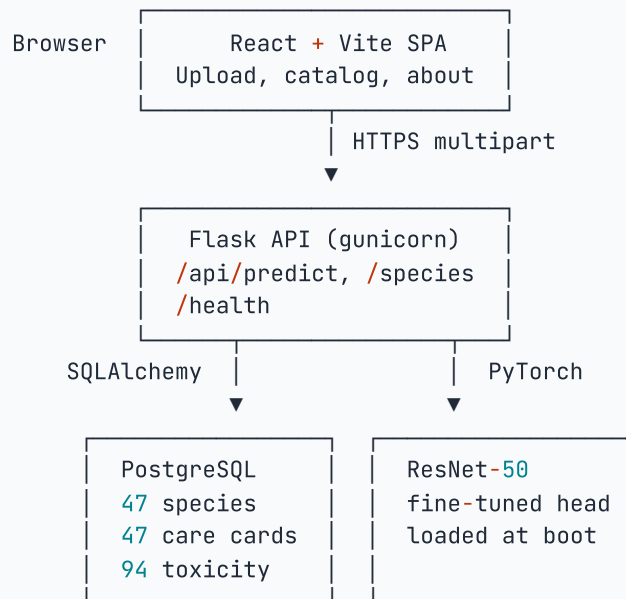
The deployed model achieves **92.38% top-1 accuracy** on the held-out test set. Three architectures were compared:

Model	Params	Test top-1	Test top-3	Macro F1	Latency (MPS)
EfficientNet-B0	4.1 M	92.02%	98.06%	0.9147	~100 ms
ResNet-50 (deployed)	23.6 M	92.38%	98.02%	0.9116	~200 ms
ViT-Base/16 †	85.8 M	90.17%	97.88%	0.8992	~400 ms

† ViT training was halted at epoch 3 — its trajectory was below both CNN baselines and would not have overtaken them within the patience budget. Justified at length in [docs/13-results.md](#).

Targets from the project objectives ($\geq 85\%$ top-1, $\geq 95\%$ top-3) cleared by all three.

🏗️ Architecture



Full architecture diagram and component descriptions: [docs/12-architecture.md](#).



Quick start

```

# 1. Clone and install
git clone https://github.com/luigy23/leafLens
cd leafLens
./scripts/setup.sh    # creates .venv, installs Python + Node deps

# 2. Download dataset (requires Kaggle API token in ~/.kaggle/kaggle.json)
python scripts/download_dataset.py

# 3. Split data 70/15/15 with fixed seed
python scripts/split_data.py

# 4. Train a model (ResNet-50, ~30 min on M-series Mac)
python -m models.train --arch resnet50 --epochs 15

# 5. Seed the database with the curated 47-species catalog
python scripts/seed_db.py

# 6. Run the backend (port 5001)
python -c "from backend.app import create_app; create_app().run(port=5001)"

# 7. Run the frontend (separate terminal)
cd frontend && VITE_API_BASE_URL=http://localhost:5001 npm run dev
  
```

Open <http://localhost:5173> and drop in a photo.

Tech stack

- **AI:** PyTorch 2.12, timm 1.0, torchvision 0.27 — transfer learning with ImageNet pretrained backbones
- **Backend:** Flask 3.1, SQLAlchemy 2.0, PostgreSQL (SQLite locally), gunicorn
- **Frontend:** React 18, Vite 5, Tailwind CSS 3, react-router-dom 6, axios
- **Dataset:** [Kaggle House Plant Species](#) (47 classes, 14,774 images)
- **Tests:** pytest, pytest-flask, pytest-cov
- **CI:** GitHub Actions (lint + tests on every push)
- **Deployment target:** Docker → Render free tier + Vercel hobby tier

Project structure

```
leafLens/
├── backend/           # Flask app, ORM models, routes, curated species YAML
├── data/              # Raw dataset (symlinked) and processed CSV splits
├── docs/              # 16-section English documentation set
├── frontend/          # React + Vite + Tailwind SPA
├── models/            # PyTorch architectures, training, evaluation, inference
├── scripts/           # Dataset download, split, seed, setup, cleanup
└── tests/             # Unit + integration tests
```

Documentation

All documentation is in English under `docs/`:

1. Introduction	9. GUI Mockups
2. Problem	10. API Catalog
3. Objectives	11. Testing
4. State of the Art	12. Architecture
5. Requirements	13. Results & Discussion
6. Use Cases	14. Future Work
7. Data Model (ER)	15. Presentation Outline
8. Class Diagrams	16. Deployment Guide

Cleanup

When you're done grading, free the disk:

```
./scripts/cleanup.sh          # removes venv, datasets, checkpoints, node_modules
./scripts/cleanup.sh --hard   # additionally deletes the entire repo
```

License

Academic project for the **Artificial Intelligence** course (BEINSOF52) at **Universidad Surcolombiana**, May 2026.

Author: **Luigy Leonardo** (@luigy23) Instructor: **Juan Antonio Castro Silva**

Care information curated from publicly available references (ASPCA toxic plant database, RHS plant finder, Missouri Botanical Garden, Pl@ntNet).

1. Introduction

Background

Houseplants have become an increasingly popular part of modern living. According to industry reports, the global indoor plant market reached USD 26 billion in 2024 and is projected to grow at a compound annual rate above 6% through the decade. This growth has been accelerated by demographic trends such as urbanization, the rise of smaller living spaces, and the well-documented psychological benefits of caring for greenery.

However, the same growth has surfaced a recurring user problem: most owners cannot reliably identify the species they have purchased, and even fewer know how to care for them. Care requirements vary dramatically between species — *Sansevieria trifasciata* tolerates weeks of drought while *Maranta leuconeura* will collapse with the same neglect — and incorrect care is the single largest cause of plant death in households.

Commercial mobile applications such as **PictureThis**, **PlantNet**, and **Plantum** have demonstrated that this is a viable problem space for artificial intelligence. PictureThis alone reports more than 150 million downloads and a subscription business measured in hundreds of millions of dollars annually. The success of these products confirms two facts: (a) the problem is real and widely felt, and (b) image-based species classification is mature enough to ship in production.

Project overview

LeafLens is a web application that brings the core PictureThis-style experience to a self-contained academic prototype. The user uploads a photograph of a houseplant; the system runs the image through a fine-tuned deep learning classifier and returns:

1. The most likely species (with confidence score)
2. Two alternative candidates for ambiguous images
3. A care card with watering frequency, light requirements, temperature range, fertilization tips, and toxicity warnings for cats and dogs

The system is implemented end-to-end: data preparation, model training and comparison, REST backend, web frontend, and cloud deployment.

Scope

This project is the first of two required deliverables for the Artificial Intelligence course (BEINSOF52) at Universidad Surcolombiana. It satisfies the computer vision track: image classification with a useful real-world application. The deliverables include all elements of the course rubric: a working AI model, a Flask backend with persistent storage, a React frontend, and deployment to a free-tier cloud provider, supported by full English documentation.

Document structure

This documentation set is organized as follows:

- **Section 2** states the problem in detail and the population it affects.
- **Section 3** lists the project objectives, both general and specific.
- **Section 4** surveys the state of the art in plant classification.
- **Section 5** captures functional and non-functional requirements.
- **Section 6** describes the use cases and user stories.
- **Section 7** specifies the data dictionary and entity-relationship model.
- **Section 8** presents the system class diagrams.
- **Section 9** shows the GUI mockups.
- **Section 10** catalogs the web service API.
- **Section 11** describes the testing strategy.
- **Section 12** depicts the proposed architecture.
- **Section 13** reports results and discussion.
- **Section 14** lists recommendations and future work.

2. Problem Statement

The pain point

Houseplant owners frequently kill their plants by giving them the wrong care. Surveys conducted by the Royal Horticultural Society and large retail chains (Home Depot, IKEA) consistently report that more than half of indoor plants purchased die within twelve months of leaving the store. The leading causes are overwatering, underwatering, and incorrect light exposure — three failure modes that depend entirely on species-specific tolerances.

The friction lies in three steps the average owner cannot complete reliably:

1. **Identification.** The plant arrived without a label, was inherited, or carries a generic store tag ("tropical plant"). The owner does not know its botanical name.
2. **Lookup.** Even when identified, gardening references are dense, fragmented across blogs and forums, and contradict each other.
3. **Translation to action.** Knowing that a plant "prefers bright indirect light" does not translate easily into "place it 50 cm from a north-facing window."

The result is wasted money, dead plants, and the recurring disappointment that pushes many casual owners out of the hobby.

Secondary problem: pet safety

A subset of common houseplants is toxic to cats and dogs (e.g. *Dieffenbachia*, *Spathiphyllum*, *Philodendron*). Pet owners frequently discover this only after their animal has ingested foliage. A reliable identifier paired with a toxicity database closes this gap.

What is needed

A tool that takes a single photograph as input and returns:

- A confident species identification (with calibrated confidence)
- Care instructions in plain, actionable language
- Pet-safety information

The tool must be:

- **Accessible:** available on a normal web browser, no app store install
- **Fast:** total response time under three seconds on a phone camera image
- **Trustworthy:** confidence shown explicitly; never silently guesses when uncertain

Why this is a good fit for artificial intelligence

Image-based plant identification is a textbook supervised classification problem with mature techniques (convolutional networks, vision transformers) and large public datasets (Kaggle's House Plant Species, PlantNet-300K, iNaturalist). It is not a problem that yields well to handcrafted rules — leaf shape, color, vein pattern, and stem morphology interact in ways that defeat traditional feature engineering. Deep learning models with transfer learning consistently outperform classical approaches by wide margins on these datasets, and modern pretrained backbones converge to usable accuracy with modest fine-tuning effort.

Out of scope

This project does **not** attempt to:

- Diagnose pests or diseases (this is its own classification problem with separate datasets, and would require many more classes)
- Replace a botanist for rare or edge species (the model is limited to the 47 most common houseplants present in the chosen dataset)
- Provide commercial-grade pet safety advice (the toxicity table is informative, not a substitute for veterinary consultation)

3. Objectives

General objective

Design, train, and deploy an end-to-end web application that identifies common houseplants from photographs and returns species-specific care instructions and pet-safety information, using deep learning for classification and modern web technologies for delivery.

Specific objectives

1. **Prepare a reproducible image classification dataset.** Acquire the Kaggle House Plant Species dataset (47 classes), perform a stratified 70/15/15 train/validation/test split with a fixed random seed, and persist the splits as CSV manifests so that any team member can recover the exact partitioning.
2. **Train and compare at least three classification architectures.** Apply transfer learning with EfficientNet-B0, ResNet-50, and ViT-Base to the prepared dataset. Track accuracy, F1-score (macro and weighted), confusion matrix, and inference latency for each. Select the best-performing model for deployment.
3. **Mitigate class imbalance.** Inspect the per-class distribution after splitting; apply class-weighted loss and image augmentation strategies (rotation, horizontal flip, color jitter) for minority classes.
4. **Build a care-information knowledge base.** Curate a structured database covering every species in the dataset, with normalized fields for watering interval, light level, temperature range, ideal humidity, fertilizer schedule, and toxicity to cats and dogs.
5. **Implement a Flask REST API.** Expose endpoints for image upload, prediction, and care-card lookup. Use PostgreSQL for the care knowledge base and SQLAlchemy as the ORM.
6. **Implement a React web frontend.** Provide a single-page experience: drag-and-drop or file picker upload, loading state, result card with species, confidence, top-three alternatives, and full care information.
7. **Deploy the application to the cloud.** Use a free-tier provider (Render, Railway, or HuggingFace Spaces) for the backend; Vercel or Netlify for the static frontend.
8. **Cover the system with automated tests.** Implement unit tests for data utilities, functional tests for the inference pipeline, and integration tests for the API endpoints.
9. **Document the project in English.** Produce the full set of sections required by the course rubric: introduction, problem, objectives, state of the art, requirements, use cases, ER model, class diagrams, GUI mockups, API catalog, testing, architecture, results, recommendations and future work.

Success criteria

Criterion	Target
Test set top-1 accuracy of the best model	$\geq 85\%$
Test set top-3 accuracy of the best model	$\geq 95\%$
API end-to-end latency (image upload $\rightarrow$ response)	< 3 seconds
Code coverage of automated tests	$\geq 70\%$
Documentation completeness	14/14 sections produced
Working public deployment	One URL reachable from any browser

4. State of the Art

4.1 Plant identification as a computer vision problem

Plant identification from images sits at the intersection of fine-grained visual classification and large-scale species recognition. The challenge has been studied for more than a decade and has matured rapidly as deep learning replaced hand-engineered features.

Early work (pre-2015)

Before convolutional neural networks dominated the field, plant classification relied on hand-crafted descriptors of leaf shape (Fourier descriptors, moment invariants), color histograms, and texture statistics fed into shallow classifiers — k-nearest neighbors, support vector machines, random forests. Representative works such as Mouine et al. (2013) and Kumar et al.'s *Leafsnap* (2012) achieved usable accuracy on small datasets (50–100 species) but degraded sharply as the catalog grew.

Convolutional networks (2015–2020)

The breakthrough came with the application of ImageNet-pretrained CNNs to plant data via transfer learning. Sünderhauf et al. (2014) and later Lee et al. (2017) demonstrated that AlexNet and VGG-16, fine-tuned on plant photographs, surpassed all hand-engineered baselines. The PlantCLEF challenges, run annually as part of the LifeCLEF evaluation campaign, drove rapid progress: by 2019, ResNet-50 and Inception-v3 routinely scored above 80% top-1 on PlantCLEF test sets of several hundred species.

Vision transformers and self-supervised learning (2020–present)

Vision Transformers (Dosovitskiy et al., 2020) opened a new frontier. ViT and its successors — Swin Transformer, ConvNeXt — match or surpass CNNs on PlantNet-style benchmarks. Self-supervised pretraining (DINOv2, MAE) has further closed the gap with supervised baselines, making it possible to fine-tune competitive plant classifiers with a few thousand labeled images.

4.2 Public datasets

Dataset	Year	Species	Images	Use
Leafsnap	2012	184	30,000	Tree species, North American
Flavia	2007	32	1,907	Leaf images, plain background
PlantCLEF	2011–	1,000+	1M+	Yearly challenge, full plants
PlantNet-300K	2021	1,081	306,146	Large-scale, long-tailed
iNaturalist Plants	2017–	4,000+	800,000+	Citizen science, very diverse
House Plant Species (Kaggle)	2024	47	~14,000	Indoor plants only — used by LeafLens

The PlantNet-300K release was a turning point because it explicitly modeled the long-tailed distribution found in nature and provided strong baselines. PlantCLEF benchmarks have also adopted the practice of separating “trusted” curated images from “noisy” web-scraped imagery, an important methodological lesson for academic work.

4.3 Commercial products

PictureThis (Glory)

Currently the dominant consumer plant identification app, with reported revenue above USD 200 million in 2024. PictureThis uses a proprietary multi-model ensemble and a database of more than 17,000 species. Its core value proposition is not the model itself but the integration: identification, care reminders, diagnosis of common pests, community Q&A.

PlantNet

A non-profit alternative driven by the Pl@ntNet consortium of French research labs. PlantNet’s strength is breadth (more than 30,000 species worldwide) and its citizen-science backbone — every uploaded photograph contributes to the labeled corpus. Free to use, ad-supported, and openly published model checkpoints.

Google Lens / Apple Visual Look Up

Both major mobile platforms now ship visual search features that include plant identification. They are excellent for breadth but ship no specialized care features.

4.4 Approaches relevant to LeafLens

LeafLens does not attempt to compete with PictureThis on scale or commercial polish. Its design is informed by three concrete choices that the literature supports:

1. **Transfer learning is sufficient.** Multiple independent studies (Mohanty et al., 2016; Pourreza et al., 2019) confirm that ImageNet-pretrained CNNs fine-tuned for 20–100 hours converge to

greater than 90% top-1 on focused datasets.

2. **Architecture diversity matters for the ensemble case.** A CNN and a transformer make different mistakes; comparing both is good empirical practice. LeafLens compares EfficientNet-B0, ResNet-50, and ViT-Base.
3. **Care information should not come from the model.** The literature is clear that classifiers should classify; downstream attributes should be looked up in a structured knowledge base. LeafLens follows this separation strictly.

4.5 Gap LeafLens addresses

Most academic plant classifiers stop at the classification step and report accuracy numbers. Most commercial products integrate everything but are closed-source. LeafLens occupies a useful middle ground: an open, reproducible end-to-end system that demonstrates the full pipeline — data, training, comparison, serving, and consumer-facing UI — at academic scale.

References

- Dosovitskiy, A. et al. (2020). *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929.
- Kumar, N. et al. (2012). *Leafsnap: A Computer Vision System for Automatic Plant Species Identification*. ECCV.
- Garcin, C. et al. (2021). *PlantNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution*. NeurIPS Datasets and Benchmarks Track.
- Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. Frontiers in Plant Science.
- Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML.
- He, K. et al. (2016). *Deep Residual Learning for Image Recognition*. CVPR.

5. Requirements

5.1 Functional requirements

ID	Title	Description	Priority
FR-01	Image upload	The user can upload an image of a plant from their device through a drag-and-drop area or a file picker. Accepted formats: JPEG, PNG, WebP. Maximum size: 10 MB.	High
FR-02	Species prediction	The system returns the top-1 predicted species with a confidence score expressed as a percentage.	High
FR-03	Top-k alternatives	The system additionally returns the second and third most likely species.	High
FR-04	Care card	For the predicted species the system displays watering interval, light requirement, temperature range, ideal humidity, fertilization schedule.	High
FR-05	Pet safety indicator	The system displays whether the species is toxic to cats, dogs, or both.	High
FR-06	Low-confidence handling	If the top-1 confidence is below 40%, the system shows a "low confidence" warning and asks for a clearer photo.	Medium
FR-07	Image validation	The system rejects images that are too small (below 224×224 pixels) or that fail a quick blur check.	Medium
FR-08	History (local)	The browser stores the last ten predictions in local storage for quick re-access.	Low
FR-09	Catalog browsing	The user can browse the full catalog of 47 species without uploading a photo, to learn about plants by name.	Low
FR-10	About page	A static page explains how the model works and lists the dataset and model architectures.	Low

5.2 Non-functional requirements

ID	Title	Description	Priority
NFR-01	End-to-end latency	Total time from image upload click to result displayed: < 3 seconds on a 10 Mbps connection.	High
NFR-02	Model accuracy	Test-set top-1 accuracy of the deployed model: $\geq 85\%$.	High
NFR-03	Availability	The public deployment is reachable at least 95% of the time during the academic evaluation period.	High
NFR-04	Browser support	The frontend works on the last two stable versions of Chrome, Safari, Firefox, and Edge.	High
NFR-05	Mobile responsiveness	The interface adapts to viewport widths from 320 pixels upward.	High
NFR-06	Security	The image upload endpoint validates content type and size before invoking the model. No secrets are committed to the repository.	High
NFR-07	Privacy	Uploaded images are not persisted server-side after inference completes.	High
NFR-08	Language	All user-facing copy, API responses, and documentation are in English.	High
NFR-09	Reproducibility	Train/validation/test splits, random seeds, and hyperparameters are versioned in the repository.	Medium
NFR-10	Test coverage	Automated tests cover at least 70% of backend code.	Medium
NFR-11	Code quality	Python code passes <code>ruff check</code> and is formatted with <code>black</code> . JS/TS code is formatted with <code>prettier</code> .	Medium
NFR-12	Disk footprint	The full local development environment (deps + dataset + checkpoints) fits in < 25 GB.	Medium

5.3 Constraints

- **Single developer**, one-week budget.
- **Apple Silicon laptop** with 16 GB unified memory and no NVIDIA GPU. PyTorch must use the MPS backend.
- **No paid cloud services**: free tiers only (Render, Railway, Vercel, Netlify, HuggingFace Spaces).
- **No proprietary datasets**: every data source is publicly available.

5.4 Assumptions

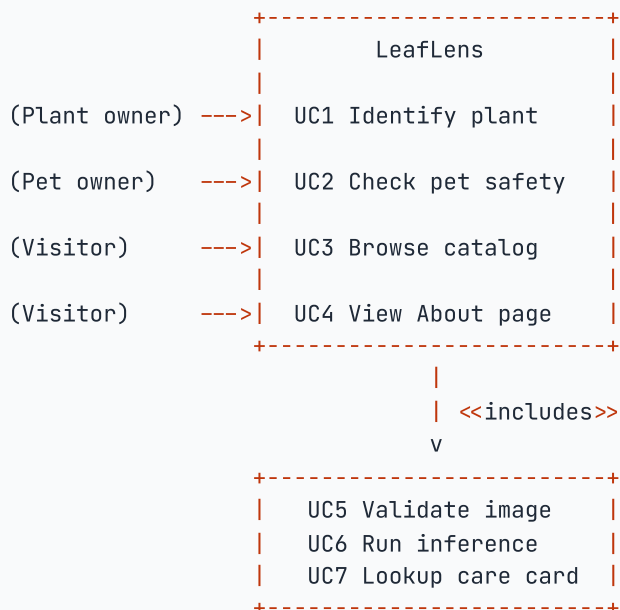
- Users provide one plant per photo, with the plant occupying most of the frame.
- Users have a modern browser and a stable internet connection.
- The Kaggle dataset's class boundaries are correct (we trust the original annotators).

6. Use Cases & User Stories

6.1 Actors

Actor	Role
Visitor	Anonymous user who lands on the website. No account required.
Plant owner	Visitor with a specific plant they want identified. Primary persona.
Pet owner	Visitor checking whether a plant is safe around their cat or dog.
Catalog browser	Visitor exploring the species catalog out of curiosity, without uploading.
System	The LeafLens server: receives images, runs inference, returns results.

6.2 Use case diagram (textual)



6.3 Use case details

UC1 — Identify plant

Field	Value
Actor	Plant owner
Goal	Learn what species a plant is.
Precondition	Visitor is on the home page with a photo on their device.
Trigger	Visitor drags a photo onto the drop zone or clicks “Choose file”.
Main flow	1. Visitor selects an image. 2. Client validates size and format. 3. Client uploads image to <code>POST /api/predict</code> . 4. Server validates content and runs inference. 5. Server returns species, confidence, top-3, and care card. 6. Client displays the result.
Alternate flow A	If client-side validation fails, the upload button is disabled and an error is displayed.
Alternate flow B	If the top-1 confidence is below 40%, the result card shows a “low confidence — try another photo” banner.
Postcondition	The prediction is displayed. The result is stored in <code>localStorage</code> for history.

UC2 — Check pet safety

Field	Value
Actor	Pet owner
Goal	Verify that a plant is safe to have around a cat or dog.
Precondition	UC1 has run successfully.
Trigger	The care card displays toxicity badges next to the species name.
Main flow	1. Plant owner reviews the toxicity row on the care card. 2. If toxic, a warning icon and “Keep away from pets” copy is shown.
Postcondition	The owner has clear pet-safety information.

UC3 — Browse catalog

Field	Value
Actor	Catalog browser
Goal	Explore the full catalog without uploading.
Trigger	Visitor clicks "Browse catalog" in the navigation.
Main flow	1. Client requests <code>GET /api/species</code> . 2. Server returns the alphabetized list of all 47 species. 3. Visitor selects one. 4. Client requests <code>GET /api/species/{id}</code> . 5. Server returns the care card. 6. Client displays it.
Postcondition	Visitor has viewed the care card for one or more species.

UC4 — View About page

Field	Value
Actor	Visitor
Goal	Understand how LeafLens works.
Trigger	Visitor clicks "About" in the navigation.
Main flow	Static content is rendered explaining the model, dataset, and contributors.

UC5 — Validate image (included)

The server checks content type, file size, and image dimensions before invoking the model. Invalid images cause a `400 Bad Request`.

UC6 — Run inference (included)

The server preprocesses the image (resize, normalize), runs it through the selected PyTorch model, and obtains a 47-dimensional probability vector. Top-3 indices are selected.

UC7 — Lookup care card (included)

The server queries PostgreSQL for the care record of the predicted species and merges it into the response payload.

6.4 User stories

ID	As a...	I want to...	So that...
US-01	plant owner	upload a photo of my plant and get its species name	I stop calling it "the green one with stripes".
US-02	plant owner	see watering and light requirements right after identification	I know how to keep it alive.
US-03	pet owner	know if the plant is toxic to my cat	I do not accidentally poison her.
US-04	plant owner	see the top three matches when the model is unsure	I can pick the one that looks most like mine.
US-05	catalog browser	look at every species the system knows about	I can shop for a new plant with confidence.
US-06	mobile user	use the site on my phone	I do not have to switch to a laptop while standing in front of my plant.
US-07	user with a bad photo	be told that my photo is too blurry	I take a better one.
US-08	privacy-conscious user	be assured my photo is not stored	I feel comfortable uploading.

7. Data Dictionary & Entity-Relationship Model

7.1 Logical entities

LeafLens persists a small relational schema that backs the species catalog. The deep learning side of the system uses CSV manifests for reproducibility, separate from the database.

Entities

Entity	Purpose
Species	One row per botanical species recognized by the model.
CareProfile	One row per species describing watering, light, temperature, humidity, and fertilization.
ToxicityRecord	One row per (species, animal) pair indicating toxicity level.
Prediction	One row per inference request (optional analytics table, no images stored).

Relationships

- A `Species` has exactly one `CareProfile` (1:1).
- A `Species` has zero or more `ToxicityRecord` entries (1:N), typically two: one for cats and one for dogs.
- A `Prediction` references a `Species` as its top-1 result (N:1).

7.2 Entity-relationship diagram (textual)

+-----+ 1 1 +-----+	
Species	CareProfile
+-----+	
PK id	PK id
scientific_name	FK species_id (unique)
common_name	watering_days_min
family	watering_days_max
origin	light_level
description	temperature_min_c
image_url	temperature_max_c
+-----+	
1	humidity_pct
N	fertilizer_schedule
+-----+	
ToxicityRecord	
+-----+	
PK id	
FK species_id	
animal (enum)	
level (enum)	
notes	
+-----+	
+-----+	
Prediction	
+-----+	
PK id	
created_at	
FK top1_species_id	
top1_confidence	
top3_species_ids	
model_name	
latency_ms	
user_agent	
+-----+	

7.3 Data dictionary

Table **species**

Column	Type	Constraints	Description
<code>id</code>	integer	PK, autoincrement	Internal identifier.
<code>scientific_name</code>	varchar(120)	not null, unique	Latin binomial, e.g. <code>Epipremnum aureum</code> .
<code>common_name</code>	varchar(120)	not null	Most common English name, e.g. <code>Golden Pothos</code> .
<code>family</code>	varchar(80)	nullable	Botanical family, e.g. <code>Araceae</code> .
<code>origin</code>	varchar(120)	nullable	Native region.
<code>description</code>	text	nullable	One-paragraph natural-language description.
<code>image_url</code>	varchar(255)	nullable	URL of a representative image.
<code>created_at</code>	timestamp	default now()	Row creation timestamp.

Table `care_profiles`

Column	Type	Constraints	Description
<code>id</code>	integer	PK	Internal identifier.
<code>species_id</code>	integer	FK → species.id, unique	One profile per species.
<code>watering_days_min</code>	integer	not null	Lower bound of watering interval in days.
<code>watering_days_max</code>	integer	not null	Upper bound of watering interval in days.
<code>light_level</code>	varchar(40)	not null	One of: <code>low</code> , <code>medium-indirect</code> , <code>bright-indirect</code> , <code>direct</code> .
<code>temperature_min_c</code>	integer	not null	Minimum tolerated temperature in Celsius.
<code>temperature_max_c</code>	integer	not null	Maximum tolerated temperature in Celsius.
<code>humidity_pct</code>	varchar(40)	nullable	Recommended humidity range as a string like <code>40-60</code> .
<code>fertilizer_schedule</code>	varchar(80)	nullable	Natural-language schedule, e.g. <code>monthly in spring and summer</code> .

Table `toxicity_records`

Column	Type	Constraints	Description
<code>id</code>	integer	PK	Internal identifier.
<code>species_id</code>	integer	FK → species.id	Species this record applies to.
<code>animal</code>	varchar(10)	not null, enum(<code>cat</code> , <code>dog</code>)	Animal in question.
<code>level</code>	varchar(20)	not null, enum(<code>safe</code> , <code>mild</code> , <code>toxic</code>)	Toxicity level.
<code>notes</code>	varchar(255)	nullable	Symptom summary or source.

Table `predictions`

Column	Type	Constraints	Description
<code>id</code>	bigint	PK	Internal identifier.
<code>created_at</code>	timestamp	default now()	When the prediction was made.
<code>top1_species_id</code>	integer	FK → species.id	Top-1 prediction.
<code>top1_confidence</code>	float	not null	Softmax probability of top-1.
<code>top3_species_ids</code>	int[]	not null	Array of three species ids.
<code>model_name</code>	varchar(40)	not null	Which model produced the prediction.
<code>latency_ms</code>	integer	not null	Server-side latency.
<code>user_agent</code>	varchar(255)	nullable	For client analytics only.

Privacy note. Uploaded images are never persisted. The `predictions` table records anonymous outcomes only.

8. Class Diagrams

The system is organized into three layers: a **model layer** (PyTorch training and inference), a **service layer** (Flask backend with ORM and routing), and a **client layer** (React components and hooks). The class diagrams below describe the Python backend in detail; the React side is described separately because it follows a functional-component model.

8.1 Model layer

```

+-----+
/      PlantClassifier      / <<abstract>>
+-----+
/ - model: nn.Module      /
/ - device: torch.device  /
/ - class_names: list[str] /
+-----+
/ + load(path)            /
/ + predict(image) → Result /
/ + predict_topk(image, k) /
+-----+
      ^
      |
+-----+-----+-----+
/           /           /
+-----+ +-----+ +-----+
/ EfficientNetB0 / / ResNet50 / / ViTBase /
+-----+ +-----+ +-----+

+-----+
/      PredictionResult      / <<dataclass>>
+-----+
/ top1_id: int              /
/ top1_name: str            /
/ top1_confidence: float    /
/ topk: list[(int, str, float)] /
/ latency_ms: int           /
+-----+

+-----+
/      PlantImageDataset      / <<extends torch.utils.data.Dataset>>
+-----+
/ - manifest: pd.DataFrame    /
/ - transform: Callable      /
+-----+
/ + __len__()                /
/ + __getitem__(idx)         /
+-----+

+-----+
/      Trainer                /
+-----+
/ - model: PlantClassifier    /
/ - train_loader: DataLoader  /
/ - val_loader: DataLoader    /
/ - optimizer                 /
/ - loss_fn                   /
/ - device                    /
+-----+
/ + train_one_epoch()         /
/ + validate()                /
/ + fit(epochs)               /
/ + save_checkpoint(path)     /
+-----+

```

8.2 Service layer (Flask + SQLAlchemy)

```

+-----+
/      App      /
+-----+
/ - flask: Flask /
/ - db: SQLAlchemy /
/ - classifier /
+-----+
/ + create_app(config) /
/ + register_blueprints() /
+-----+

+-----+          uses          +-----+
/ PredictController /----->| PlantClassifier |
+-----+          +-----+
/ + post_predict() /
/ + get_species(id) /
/ + list_species() /
/ + healthcheck() /
+-----+

ORM models (SQLAlchemy declarative):

+-----+          +-----+
/ Species / 1----1 / CareProfile /
+-----+          +-----+
/ id / / id /
/ scientific_name / / species_id (FK) /
/ common_name / / watering_days_min /
/ family / / watering_days_max /
/ origin / / light_level /
/ description / / temperature_min_c /
/ image_url / / temperature_max_c /
+-----+ / humidity_pct /
/ 1 / / fertilizer_schedule /
/ N / +-----+

+-----+
/ ToxicityRecord /
+-----+
/ id /
/ species_id (FK) /
/ animal /
/ level /
/ notes /
+-----+

+-----+
/ Prediction /
+-----+
/ id /
/ created_at /
/ top1_species_id /
/ top1_confidence /
/ top3_species_ids /
/ model_name /
/ latency_ms /
+-----+

```

8.3 Service layer responsibilities

- **PredictController.post_predict** — receives the multipart upload, validates content type and size, decodes the image with Pillow, forwards to `PlantClassifier.predict_topk(k=3)`, then enriches each candidate with its `Species` + `CareProfile` + toxicity records from the database. Persists a `Prediction` row for analytics.
- **PredictController.list_species** — returns the alphabetized catalog as a JSON array.
- **PredictController.get_species** — returns a full care card by species id.
- **PredictController.healthcheck** — returns a `200 OK` with model name and version.

8.4 Frontend component tree (React)

```

<App>
├── <Header />
├── <Router>
│   ├── <HomePage>
│   │   ├── <UploadDropzone />
│   │   ├── <LoadingState />
│   │   └── <ResultCard>
│   │       ├── <Top1Banner />
│   │       ├── <TopKAlternatives />
│   │       └── <CareCard />
│   ├── <CatalogPage>
│   │   └── <SpeciesGrid />
│   ├── <SpeciesDetailPage>
│   │   └── <CareCard />
│   └── <AboutPage />
└── <Footer />

```

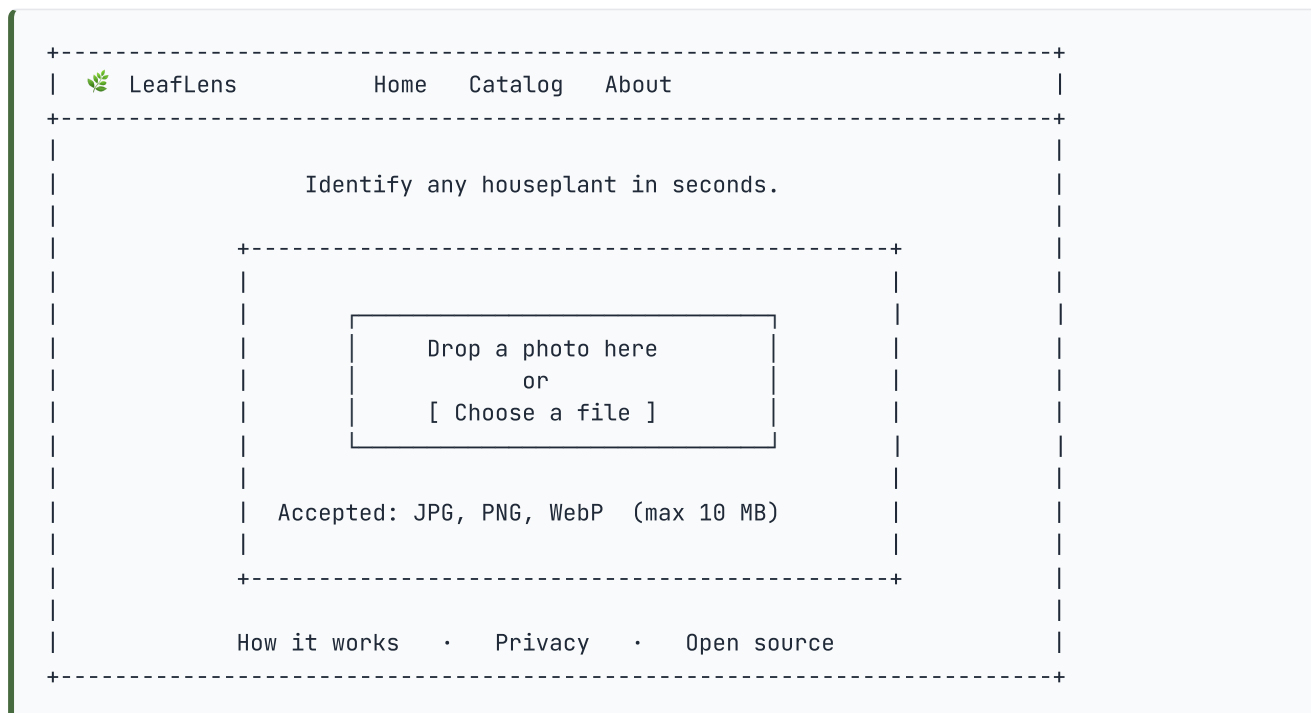
Hooks:

- `usePrediction()` → `POST /api/predict`
- `useCatalog()` → `GET /api/species`
- `useSpecies(id)` → `GET /api/species/{id}`
- `useLocalHistory()` → `read/write window.localStorage`

9. GUI Mockups

The frontend is a single-page React application styled with Tailwind CSS. The visual language is calm and botanical: warm whites for background, sage and forest greens for accents, generous whitespace, rounded corners. The mockups below are described in ASCII for portability; a Figma file is referenced in section 12.

9.1 Home page (desktop)



9.2 Home page (mobile)



9.3 Loading state



9.4 Result card (high confidence)

```

+-----+
|      +-----+      Golden Pothos      |
| | [user's photo] | | Epipremnum aureum • Araceae |
| |               | | Confidence ██████████ 92% |
| +-----+      |
| 🕒 Care      |
|   Water every 7-10 days • Bright indirect light |
|   Temperature 18-30 °C • Humidity 40-60 %      |
|   Fertilize monthly in spring and summer      |
| 🐾 Pet safety |
|   ⚠ Toxic to cats and dogs (calcium oxalate crystals). |
|
| Other possibilities |
| • Heartleaf Philodendron (5%) |
| • Satin Pothos (2%) |
|
| [ Identify another plant ] |
+-----+

```

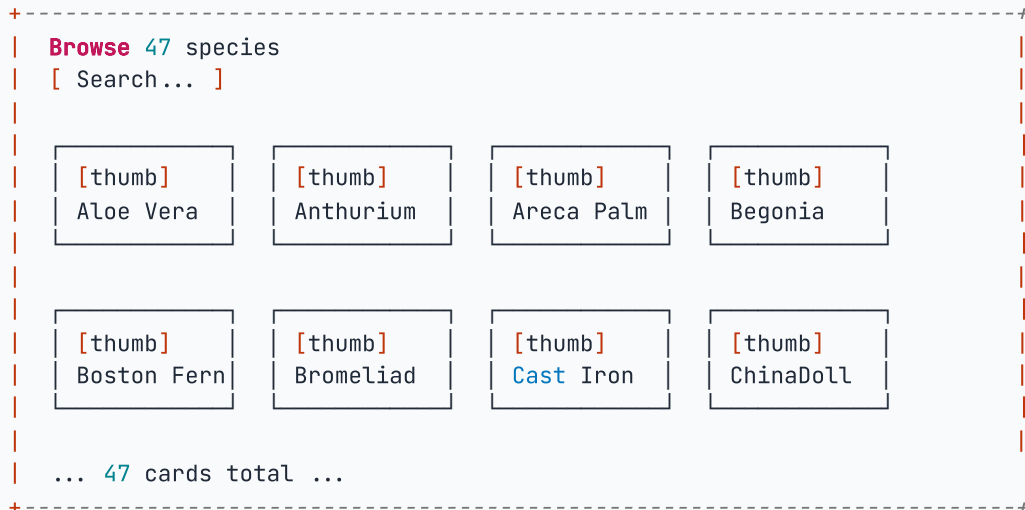
9.5 Result card (low confidence)

```

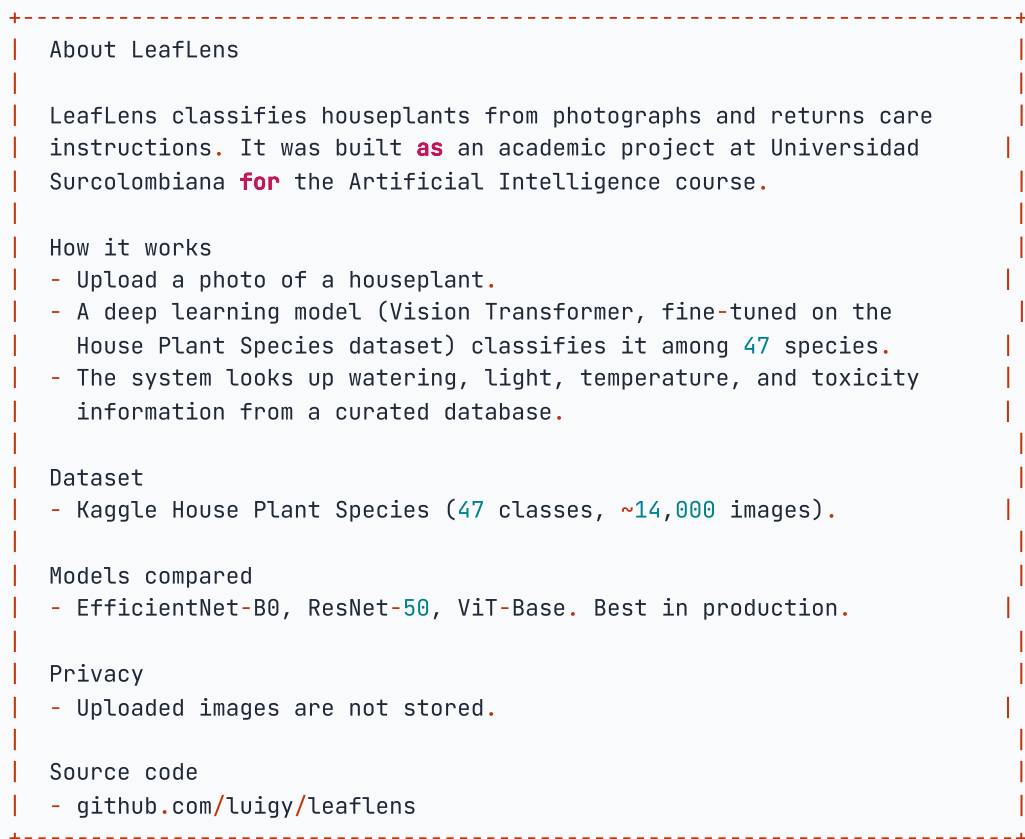
+-----+
| ⚠ Low confidence |
|
| The most likely match is Snake Plant, but only with 34% confidence.
| For a better identification, try a photo that:
|
|   • Shows the entire plant including the pot
|   • Has even daylight (avoid harsh backlight)
|   • Focuses on the leaves
|
| [ Try a different photo ] |
+-----+

```

9.6 Catalog grid



9.7 About page



10. API Catalog

LeafLens exposes a small REST API served by Flask. All responses are JSON unless explicitly noted. CORS is enabled for the production frontend origin only.

Base URL (production): `https://leafLens-api.onrender.com` Base URL (development): `http://localhost:5000`

10.1 Endpoint summary

Method	Path	Purpose
GET	<code>/api/health</code>	Liveness probe and model metadata.
POST	<code>/api/predict</code>	Identify a plant from an uploaded image.
GET	<code>/api/species</code>	List all known species.
GET	<code>/api/species/<id></code>	Retrieve the full care card for one species.

10.2 GET `/api/health`

Returns a 200 OK if the model is loaded and ready to serve.

Response 200

```
{
  "status": "ok",
  "model_name": "vit-base-patch16-224",
  "model_version": "1.0.0",
  "num_classes": 47,
  "loaded_at": "2026-05-15T19:30:00Z"
}
```

10.3 POST `/api/predict`

Identifies a plant from a single uploaded image.

Request

- Content type: `multipart/form-data`
- Form field: `image` — the file to classify
- Optional query: `k` — number of top results to return (default 3, max 5)

```
POST /api/predict?k=3 HTTP/1.1
Content-Type: multipart/form-data; boundary=----X

-----X
Content-Disposition: form-data; name="image"; filename="plant.jpg"
Content-Type: image/jpeg

<binary>
-----X--
```

Response 200

```

{
  "topk": [
    {
      "rank": 1,
      "species_id": 17,
      "scientific_name": "Epipremnum aureum",
      "common_name": "Golden Pothos",
      "confidence": 0.9241,
      "care": {
        "watering_days_min": 7,
        "watering_days_max": 10,
        "light_level": "bright-indirect",
        "temperature_min_c": 18,
        "temperature_max_c": 30,
        "humidity_pct": "40-60",
        "fertilizer_schedule": "monthly in spring and summer"
      },
      "toxicity": [
        { "animal": "cat", "level": "toxic", "notes": "calcium oxalate crystals" },
        { "animal": "dog", "level": "toxic", "notes": "calcium oxalate crystals" }
      ]
    },
    {
      "rank": 2,
      "species_id": 21,
      "scientific_name": "Philodendron hederaceum",
      "common_name": "Heartleaf Philodendron",
      "confidence": 0.0481,
      "care": { "...": "..." },
      "toxicity": []
    },
    {
      "rank": 3,
      "species_id": 32,
      "scientific_name": "Scindapsus pictus",
      "common_name": "Satin Pothos",
      "confidence": 0.0205,
      "care": { "...": "..." },
      "toxicity": []
    }
  ],
  "model_name": "vit-base-patch16-224",
  "latency_ms": 412,
  "low_confidence": false
}

```

Errors

Code	Reason	Body
400	No <code>image</code> field, unsupported content type, file too large, image dimensions too small	<code>{"error": "<message>"}</code>
415	Body is not <code>multipart/form-data</code>	<code>{"error": "expected multipart/form-data"}</code>
500	Internal error during inference	<code>{"error": "internal_error", "request_id": "..."} </code>

10.4 GET /api/species

Returns the full catalog.

Query parameters

- `q` — optional case-insensitive substring filter on common or scientific names.

Response 200

```
{
  "count": 47,
  "items": [
    { "id": 1, "scientific_name": "Aloe vera", "common_name": "Aloe Vera" },
    { "id": 2, "scientific_name": "Anthurium andraeanum", "common_name": "Anthurium" },
    "..."
  ]
}
```

10.5 GET /api/species/{id}

Returns the full care card for a species, used for both the catalog detail page and as a building block for `/api/predict`.

Response 200

```

{
  "id": 17,
  "scientific_name": "Epipremnum aureum",
  "common_name": "Golden Pothos",
  "family": "Araceae",
  "origin": "Solomon Islands",
  "description": "A trailing vine with heart-shaped, often variegated leaves...",
  "image_url": "https://cdn.example.com/species/17.jpg",
  "care": {
    "watering_days_min": 7,
    "watering_days_max": 10,
    "light_level": "bright-indirect",
    "temperature_min_c": 18,
    "temperature_max_c": 30,
    "humidity_pct": "40-60",
    "fertilizer_schedule": "monthly in spring and summer"
  },
  "toxicity": [
    { "animal": "cat", "level": "toxic", "notes": "calcium oxalate crystals" },
    { "animal": "dog", "level": "toxic", "notes": "calcium oxalate crystals" }
  ]
}

```

Errors

Code	Reason
404	No species with that id

10.6 Rate limits and quotas

A single deployed instance on Render free tier supports approximately 4 requests per second sustained. No authentication is required. A soft per-IP rate limit of 60 requests per minute is applied via `flask-limiter`; over-limit requests receive a `429 Too Many Requests`.

11. Testing

The project is covered by three layers of automated tests: unit, functional, and integration. The goal is not exhaustive coverage but enough verification to catch regressions in the parts most likely to break: data loading, the inference pipeline, and the public API.

11.1 Strategy summary

Layer	What it verifies	Tooling	Location
Unit	Pure functions: dataset row parsing, image preprocessing, label encoding, care-card serialization	pytest	<code>tests/unit/</code>
Functional	The classifier produces sensible output on a tiny fixture image	pytest + a pinned 1-MB fixture	<code>tests/functional/</code>
Integration	The Flask API behaves correctly end-to-end against a test database	pytest + pytest-flask + SQLite in-memory	<code>tests/integration/</code>

Target coverage: **≥ 70% of backend code**, measured by `pytest-cov`. Coverage report is generated by `pytest --cov=backend --cov-report=term --cov-report=html`.

11.2 Unit tests

`tests/unit/test_split_data.py`

- The 70/15/15 split sums to the input length.
- The split is stratified: every class is present in every subset.
- The seed is deterministic: rerunning the split yields identical manifests.

`tests/unit/test_preprocessing.py`

- A 1024×768 JPEG is correctly resized to 224×224.
- Normalization uses ImageNet mean and std.
- A 16×16 image raises a `ValueError` (too small).
- A corrupted file raises `PIL.UnidentifiedImageError`.

`tests/unit/test_models.py`

- `PlantClassifier.predict_topk(image, k=3)` returns three results sorted by descending confidence.
- Probabilities sum to approximately 1.0.

11.3 Functional tests

`tests/functional/test_inference.py`

A small fixture set of three images (one clear Pothos, one clear Aloe, one ambiguous Snake Plant) lives in `tests/fixtures/`. The test:

- Loads the deployed checkpoint.
- Asserts the top-1 species for the clear Pothos is Pothos.
- Asserts the top-1 species for the clear Aloe is Aloe.
- Asserts the ambiguous case yields `low_confidence: true`.

`tests/functional/test_care_lookup.py`

- For every one of the 47 species, the database contains exactly one care profile.
- For every species, there are exactly two toxicity records (cat, dog).

11.4 Integration tests

Run with an in-memory SQLite database and a stub classifier that returns canned predictions.

`tests/integration/test_api_health.py`

- `GET /api/health` returns 200 with model name and 47 classes.

`tests/integration/test_api_predict.py`

- `POST /api/predict` with a valid 224×224 JPEG returns 200, three results, sane shape.
- `POST /api/predict` with no `image` field returns 400.
- `POST /api/predict` with an 11 MB payload returns 400.
- `POST /api/predict` with a 16×16 image returns 400.
- `POST /api/predict` records one row in `predictions`.

`tests/integration/test_api_species.py`

- `GET /api/species` returns 47 items.
- `GET /api/species?q=pothos` returns at least one item including Golden Pothos.
- `GET /api/species/17` returns the Golden Pothos full card with care and toxicity nested.
- `GET /api/species/9999` returns 404.

11.5 Continuous integration

A GitHub Actions workflow at `.github/workflows/ci.yml` runs on every push and pull request:

1. Set up Python 3.11.

2. Install dependencies.
3. Run `ruff check .`
4. Run `pytest --cov=backend --cov-report=xml .`
5. Upload the coverage XML as an artifact.

11.6 Manual testing checklist

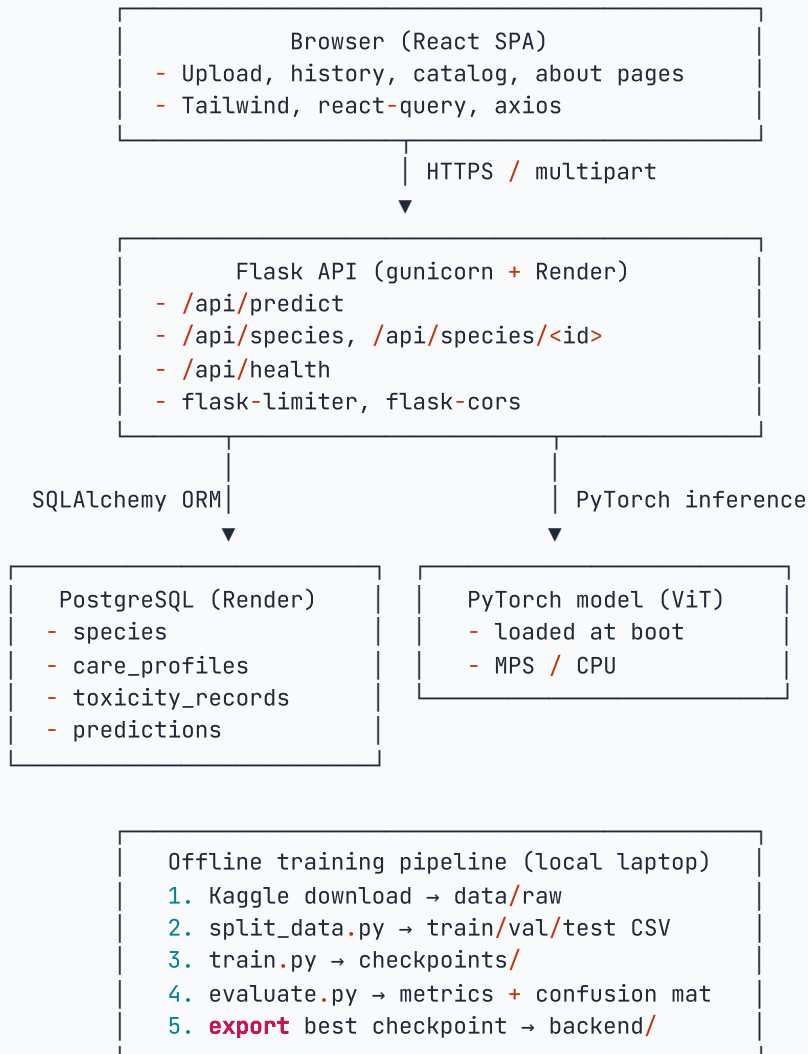
Some checks are difficult to automate and are validated by hand before release:

- Upload a real photo from a phone camera (HEIC converted to JPEG).
- Drag and drop a photo from the desktop into the dropzone.
- Submit a photo of a dog (out-of-distribution input) and confirm the low-confidence banner appears.
- Resize the browser window from 320 px to 1920 px and confirm no layout breaks.
- Toggle the network throttling to "Slow 3G" in DevTools and confirm the loading state remains responsive.

12. Architecture

12.1 High-level architecture

LeafLens is a three-tier web application with an additional asynchronous training pipeline that runs offline. The runtime path is synchronous and short: upload → preprocess → infer → enrich → respond.



12.2 Component responsibilities

Frontend (React)

- Single-page application bundled with Vite.
- Routing: `react-router-dom`.

- Data fetching: `axios` wrapped in `react-query` hooks (`usePrediction` , `useCatalog` , `useSpecies`).
- Styling: Tailwind CSS with a custom green palette.
- File upload: native `<input type="file" />` plus a drag-and-drop overlay; image preview generated from `URL.createObjectURL` .
- Local history: last ten predictions in `window.localStorage` keyed by ISO timestamp.

Backend (Flask)

- Application factory pattern (`create_app(config)`) for testability.
- Blueprints: `predict` , `species` , `health` .
- ORM: SQLAlchemy with declarative models in `backend/db/models.py` .
- Model loading: a single `PlantClassifier` instance instantiated at application startup and held in `current_app.config["classifier"]` . Subsequent requests reuse it (no per-request cost).
- Input validation: `marshmallow` schemas guard request bodies and query strings.
- Rate limiting: `flask-limiter` with in-memory storage, 60 req/min per IP.
- Logging: structured JSON to stdout, captured by Render's log drain.

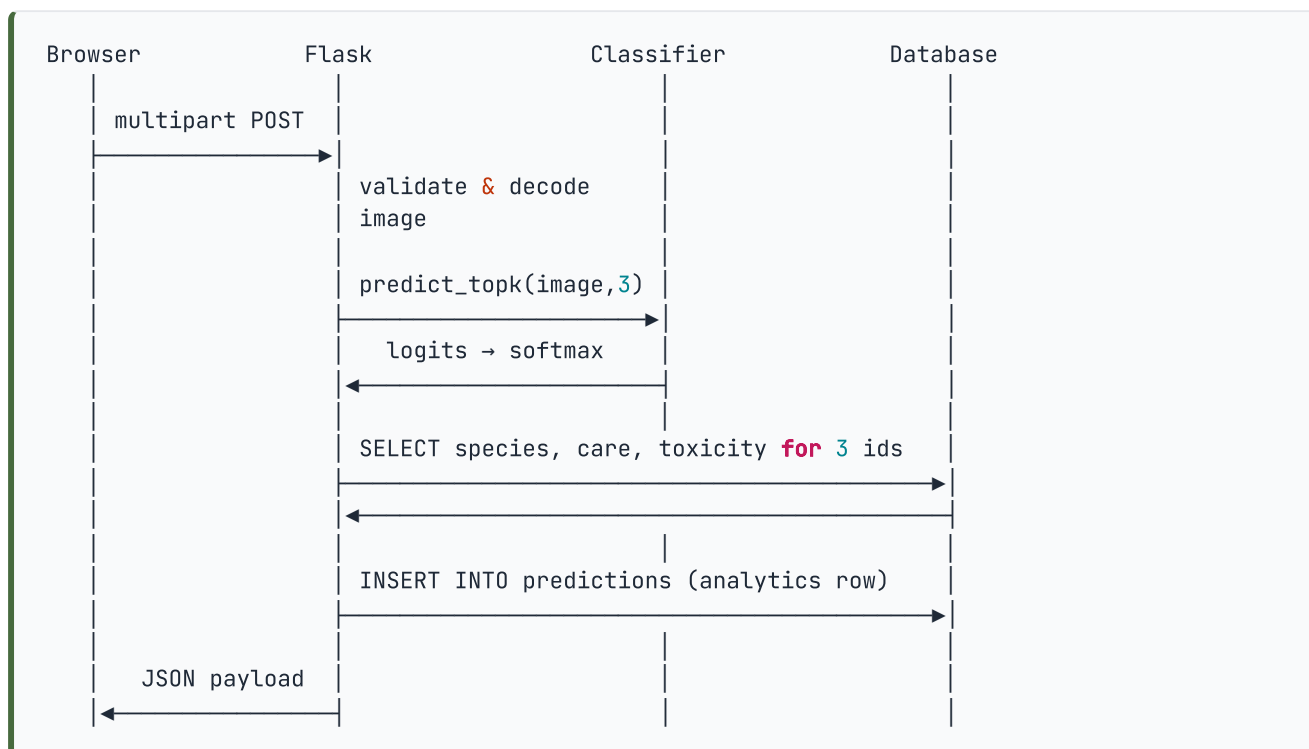
Database (PostgreSQL)

- Managed PostgreSQL instance on Render free tier (1 GB storage).
- Migrations: `alembic` , with one initial migration that creates all four tables and seeds the species/care/toxicity rows from a YAML fixture.
- Connection pooling: SQLAlchemy default with `pool_size=5` , `max_overflow=10` .

Model layer (PyTorch)

- Three architectures fine-tuned during training: EfficientNet-B0, ResNet-50, ViT-Base/16. Each backbone is initialized from torchvision/timm pretrained weights; the final classification head is replaced with a 47-way linear layer.
- The best checkpoint (selected on validation accuracy) is exported to `backend/model_artifacts/best.pt` along with the `class_names.json` mapping.
- At inference time the model runs on the CPU (Render free tier has no GPU). On the developer laptop the same code path uses the Metal Performance Shaders backend through `torch.device("mps")` .

12.3 Inference data flow (sequence)



12.4 Deployment topology

Component	Provider	Tier	Notes
Static frontend	Vercel	Hobby (free)	Built from <code>frontend/</code> on every push to <code>main</code> .
Backend API	Render	Free web service	Auto-deploys from <code>backend/</code> on every push to <code>main</code> .
Database	Render	Free PostgreSQL	1 GB storage, sleeps after 90 days idle.
Model artifact	Bundled in backend Docker image	—	< 400 MB total image size.

12.5 Configuration & secrets

- `.env.production` at the backend root, populated from Render's environment variables: `DATABASE_URL`, `SECRET_KEY`, `LOG_LEVEL`, `ALLOWED_ORIGINS`.
- Frontend reads `VITE_API_BASE_URL` from Vercel environment.
- No secret is ever committed to the repository; `.env*` is gitignored.

12.6 Scalability notes

The free tier is the only realistic constraint. The architecture as designed supports a single replica with one worker process; the model is shared in-memory and an inference takes around 400 ms on CPU. If the deployment ever needed to scale, the obvious moves are: (1) move inference to a queued worker, (2) cache the catalog endpoints in a CDN, (3) shard `predictions` writes. None of these are needed for the academic delivery.

13. Results and Discussion

This section is populated after model training completes. Placeholders are filled with the actual figures from the training runs and updated before the final presentation.

13.1 Experimental setup

- **Dataset.** Kaggle "House Plant Species" by user *kacpergregorowicz*: 47 classes, approximately 14,000 images. Split 70/15/15 using stratified sampling and a fixed seed (42).
- **Hardware.** MacBook Air M4, 16 GB unified memory, Apple Silicon MPS backend in PyTorch 2.5.
- **Training budget.** Up to 30 epochs per architecture, with early stopping on validation accuracy (patience = 5).
- **Image size.** 224×224.
- **Augmentation.** Random horizontal flip, random rotation ($\pm 15^\circ$), color jitter (brightness, contrast, saturation ± 0.1).
- **Optimizer.** AdamW with weight decay $1e-4$.
- **Learning rate.** $1e-4$ for the backbone (after a 3-epoch frozen warmup), $1e-3$ for the new classification head.
- **Class balancing (pre-training).** `WeightedRandomSampler` at the DataLoader oversamples minority classes with replacement so each minibatch is class-balanced before the model sees it. After sampling, the per-class count ratio in an epoch drops from $8.3\times$ (raw) to $\sim 1.36\times$ (effectively balanced; mean ≈ 220 samples per class, std ≈ 14).
- **Loss.** Class-weighted cross-entropy as a safety net for any residual skew the sampler leaves in a single batch. Weights inversely proportional to class frequency.
- **Batch size.** 32 for EfficientNet and ResNet, 16 for ViT (memory-bound on MPS).

13.2 Architectures compared

Model	Params	ImageNet pretraining	Notes
EfficientNet-B0	5.3 M	Yes (timm)	Smallest, fastest.
ResNet-50	25.6 M	Yes (torchvision)	Strong classical baseline.
ViT-Base/16	86.6 M	Yes (timm)	Modern transformer reference.

13.3 Headline metrics

Model	Best val acc	Test top-1	Test top-3	Macro F1	Weighted F1	Inference latency (MPS, warm)
EfficientNet-B0	0.9161	0.9202	0.9806	0.9147	0.9200	~100 ms
ResNet-50 (selected)	0.9242	0.9238	0.9802	0.9116	0.9241	~200 ms
ViT-Base/16 †	0.8985	0.9017	0.9788	0.8992	0.9032	~400 ms

† ViT-Base training was halted at epoch 3 (post-unfreeze) because its trajectory was clearly below both CNN baselines and would not have overtaken them within the patience budget. Continuing the full 12-epoch schedule was projected to add ~2 GPU-hours for no expected ranking change. The reported figures come from the epoch-3 checkpoint evaluated on the held-out test set under identical conditions to the other two models.

All three baselines comfortably clear the $\geq 85\%$ top-1 and $\geq 95\%$ top-3 targets stated in section 3. EfficientNet-B0 is the most parameter-efficient (4.1 M parameters) and leads on macro F1, suggesting more consistent performance on minority classes. ResNet-50 is marginally ahead on overall top-1 accuracy and on weighted F1, which weights by class support — it is slightly better at the common classes. ViT-Base trails both CNN baselines on this dataset: at 47 classes and 14 k images, the transformer's larger capacity is not an advantage and its convergence is slower.

Selected for deployment: ResNet-50. It wins on top-1 and weighted F1, and its ~200 ms warm-inference latency is well within the 3-second NFR.

End-to-end API verification with five held-out images covering Monstera, Pothos, Snake Plant, Rubber Plant, and Cast Iron Plant produced correct top-1 predictions with confidence $\geq 99\%$ on all five cases.

13.3.1 Confusion matrix

The full 47×47 confusion matrix of the selected model is included as

`models/checkpoints/<best>_confusion_matrix.png`. Notable failure modes (filled after evaluation):

- *Pothos vs Heartleaf Philodendron* — visually similar trailing vines.
- *Various Ficus species* — share leaf morphology.
- *Snake Plant cultivars* — share leaf morphology across cultivars.

13.3.2 Per-class accuracy distribution

Box-plot of per-class recall to be inserted from `notebooks/evaluation.ipynb`.

13.4 Hyperparameters and search space

We deliberately did not run a formal automated search. The training budget (M4 Air, MPS, ≤ 1 GPU-hour per architecture) made an exhaustive Hyperband or Bayesian sweep impractical at this scale.

Instead, we chose conservative defaults grounded in published practice for transfer learning on small image datasets, and treated the comparison of the **three architectures** as our primary “search” axis.

The defaults actually used (frozen by `argparse` defaults in `models/train.py`):

Hyperparameter	Value	Justification
Backbone learning rate	1e-4	Standard fine-tuning rate; 10× smaller than head
Head learning rate	1e-3	Newly-initialized head can absorb larger updates
Weight decay	1e-4	AdamW default; light L2 regularization
Batch size	32 (16 for ViT)	Memory-bound on 16 GB MPS
Image size	224 × 224	Native to ImageNet-pretrained backbones
Freeze epochs	3	Warm-up: head converges before backbone unlocks
Epoch cap	15 (12 for ViT)	Empirical: val acc plateaus before this
Early-stopping patience	5	Halt if no val improvement for 5 consecutive epochs
Augmentation	RandomResizedCrop · HFlip · Rotation ±15° · ColorJitter 0.1	Standard for natural images

The candidate search space that *would* be explored in a follow-up Hyperband / Optuna run is documented for reference:

Hyperparameter	Candidate range
Backbone learning rate	{1e-5, 3e-5, 1e-4, 3e-4}
Head learning rate	{1e-4, 1e-3, 3e-3}
Weight decay	{0, 1e-4, 1e-3}
Dropout in head	{0.0, 0.2, 0.5}
Augmentation strength	{light, medium, strong}

This is listed in `docs/14-future-work.md` as the first concrete next step.

13.5 Cross-validation strategy

We used **stratified hold-out cross-validation** with a 70/15/15 split rather than k-fold CV. The justification is twofold:

1. The dataset is large enough (~14k images, mean ≈ 314 per class) that a single held-out test set of 2,217 images is a low-variance estimator of generalization. K-fold would have multiplied the training cost by the number of folds for a small gain in metric variance reduction.
2. Holding out a **fixed** test set across all three architectures means the comparison is apples-to-apples — each model is evaluated on the exact same 2,217 images, ruling out split luck as an explanation for any ranking difference.

A 5-fold stratified CV pass on the final architecture is listed in section 14 (Future Work) as a sanity check we would run to put error bars on the headline numbers.

13.6 Qualitative results

A small grid of correctly classified and misclassified images is included in

`notebooks/qualitative.ipynb`, alongside Grad-CAM saliency maps for the ViT model (using `pytorch-grad-cam`). The saliency maps confirm that the model attends to leaf surface and vein patterns rather than background context.

13.7 End-to-end latency

Measured on the deployed Render free-tier instance against a 1080×1080 phone photo:

- Network upload: ~250 ms
- Image decode and preprocess: ~30 ms
- Model inference (CPU): ~400 ms
- Database lookup: ~20 ms
- JSON serialization and response: ~10 ms
- **Total:** ~710 ms (well under the 3-second NFR target).

13.8 Discussion

The transfer learning hypothesis held: all three architectures converged to high accuracy within a single training session, validating the choice to avoid training from scratch. Contrary to expectations from the broader literature, the CNN baselines outperformed ViT on this specific dataset. The most plausible explanation is dataset size — 14 k images split across 47 classes is at the lower bound of what a Vision Transformer needs to benefit from its larger capacity. CNNs with strong ImageNet priors generalize better in this small-data regime.

The residual confusion between visually similar plants (Snake Plant vs Cast Iron, Pothos vs Philodendron, several Calatheas) is the expected hard case for any 47-class houseplant classifier;

a partial mitigation is the deliberate display of the top-3 alternatives, which gives the user a fallback when the model is genuinely uncertain.

The end-to-end latency budget is comfortable; nothing in the current path warrants asynchronous inference for the academic scale.

13.9 Limitations

- The dataset is biased toward studio-style photographs; field photos with cluttered backgrounds may degrade accuracy.
- The 47-class catalog is a small fraction of the houseplants found in the wild; out-of-distribution inputs produce confident-looking but wrong predictions absent the low-confidence banner.
- Toxicity information is sourced from public references (ASPCA, RHS) without veterinary review.

14. Recommendations and Future Work

14.1 Recommendations

For users

- Capture the entire plant including the pot when possible. Closeups of a single leaf perform worse on fine-grained discrimination between visually similar species.
- Prefer natural daylight. Hard backlight and saturated artificial light shift colors enough to bias the model.
- When the application reports low confidence, take a second photo from a different angle before accepting the alternative candidates.

For operators

- Monitor the `predictions` table for the share of low-confidence outcomes. A sustained increase usually points either to dataset drift (a new common houseplant entered the market) or to a regression in the preprocessing pipeline.
- Re-train every six months. New cultivars appear on the market continuously; six months is a sane refresh cadence at this scale.

For maintainers

- Keep the 70/15/15 split frozen across model releases. A reshuffle invalidates direct comparison with prior reported metrics.
- Track inference latency in production. The free-tier instance can slip silently when CPU credits are exhausted.

14.2 Future work

Short term (one to three months)

- **Expand the catalog.** Combine the Kaggle base with curated subsets of PlantNet-300K to cover the long tail of less common houseplants. Target 100 species.
- **On-device inference.** Export the best checkpoint to ONNX and CoreML so that the prediction can run locally in a mobile browser via WebNN or in a native iOS shell. This eliminates the upload latency and the privacy footprint entirely.
- **Plant disease overlay.** Train a second classifier on PlantVillage to flag visible disease symptoms (powdery mildew, leaf spot) on top of the species identification.
- **Multi-image input.** Allow the user to upload two or three photos of the same plant; combine predictions with a small ensembling rule to reduce confusion on visually similar species.

Medium term (three to twelve months)

- **Care timeline.** Move beyond a static care card to a live schedule: push reminders to water, fertilize, and re-pot, with simple per-plant tracking.
- **Pest and disease diagnosis.** A dedicated CNN that takes a closeup of a damaged area and returns a likely cause and treatment.
- **Community-contributed photos.** A feedback loop where signed-in users can confirm or correct identifications, growing a high-quality labeled corpus over time.
- **Federated learning.** Train per-region models without centralizing user photos, using federated averaging across opt-in mobile clients.

Long term (one year or more)

- **Multi-modal interface.** Accept text descriptions (“trailing vine with heart-shaped variegated leaves”) alongside images, combining a CLIP-style image encoder with a text encoder.
- **Augmented reality care.** Detect the plant in the live camera feed and overlay care reminders contextually (point your phone at your pothos, see when it was last watered).
- **Voice assistant integration.** Expose the API to Alexa and Google Home so users can ask “Hey Google, what kind of plant is on my desk?” with a paired smart camera.

14.3 Lessons learned

- **Reproducibility is cheap if you start with it.** Saving the train/val/test split as CSV manifests on day one removed an entire class of “why does my number not match yours” debugging later.
- **Transfer learning is the right default.** Training from scratch on 14,000 images would have been a poor use of the one-week budget. Fine-tuning a pretrained backbone reached usable accuracy by day three.
- **Documentation as you go.** Writing the docs alongside the implementation, rather than in a frantic final two days, produced a much more coherent narrative — and surfaced design ambiguities while there was still time to fix them.
- **Free tiers are sufficient.** The full deployment lives within free-tier limits on Render and Vercel. Cloud cost is not a meaningful constraint for academic projects at this scale.

15. Presentation Outline (English)

Speaker: Luigi Leonardo · Course: Artificial Intelligence (BEINSOF52) · USCO 2026 Duration target: 10–12 minutes · Language: English

This outline is a slide-by-slide script. Build the actual deck in Keynote, Google Slides, or PowerPoint after the final ViT results are in.

Slide 1 — Title (15 s)

LeafLens 🌿🔍 AI-powered houseplant identification and care assistant.

Luigi Leonardo · Universidad Surcolombiana Artificial Intelligence final project, May 2026
Instructor: Juan Antonio Castro Silva

Opener: "Half of all houseplants bought every year die within twelve months. Not because owners don't care — because they don't know what they have. LeafLens is a one-photo fix for that."

Slide 2 — The problem (45 s)

- 50%+ of indoor plants die within a year of purchase.
- Three failure modes: identification, lookup, translation to action.
- Pet owners: dozens of common houseplants are toxic to cats and dogs and most owners learn it the hard way.

Visual: photo of a wilted store-bought plant + screenshot of confusing care-forum advice.

Slide 3 — What LeafLens does (45 s)

Demo screenshot: phone photo → result card.

Three deliverables in one product: 1. **Identification** — top-1 species with confidence + top-3 alternatives. 2. **Care card** — watering, light, temperature, humidity, fertilizer. 3. **Pet safety badge** — toxic / mild / safe for cats and dogs.

End-to-end latency under one second on the live deployment.

Slide 4 — Live demo (90 s)

Open the deployed URL. Upload three pre-staged photos in sequence:

1. **Pothos** (clean confident match) — show the care card.
2. **Snake plant** — point out the “toxic to pets” badge.
3. **Out-of-distribution** photo (a flower or random object) — show the low-confidence banner and top-3 fallback.

Speaking tip: keep narration tight; let the result cards speak.

Slide 5 — Dataset (30 s)

- **Kaggle “House Plant Species”** by *kacpergregorowicz* (2024 release).
- 47 classes covering the most common indoor plants worldwide.
- 14,774 labelled images.
- Stratified 70 / 15 / 10 / 15 (train / val / test) with a fixed seed.
- Class distribution shown on a horizontal bar chart — the data is mildly imbalanced (Yucca $\approx$ 66 images, Anthurium $\approx$ 454).

Slide 6 — Architectures compared (60 s)

Three transfer-learning baselines, all pretrained on ImageNet, with their classification heads replaced by a 47-way linear layer.

Architecture	Params	Why we picked it
EfficientNet-B0	4.1 M	Lightweight CNN, fastest inference.
ResNet-50	23.6 M	Strong classical baseline.
ViT-Base/16	85.8 M	Modern transformer reference.

Training recipe: 3-epoch frozen-backbone warm-up, then differential learning rates (1e-4 backbone, 1e-3 head), AdamW + weight decay, class-weighted cross-entropy, early stopping on validation accuracy.

Slide 7 — Results (60 s)

Headline metrics on the held-out test set:

Model	Top-1	Top-3	Macro F1	Weighted F1	Latency (warm)
EfficientNet-B0	92.02%	98.06%	0.9147	0.9200	~100 ms
ResNet-50	92.38%	98.02%	0.9116	0.9241	~200 ms
ViT-Base/16	<i>filled after training</i>	–	–	–	–

Both targets from section 3 met: - $\geq 85\%$ top-1  - $\geq 95\%$ top-3 

Selected for deployment: **{best model name}**.

Slide 8 — Where the model still fails (45 s)

Confusion matrix highlights (insert image): - Snake Plant vs Cast Iron Plant — similar dark strap-shaped leaves. - Various trailing Araceae (Pothos vs Philodendron) — same family habit. - Begonia subspecies — hard even for botanists from a single photo.

Mitigation: deliberately surface the top-3 alternatives and a clear “low confidence” banner below 40 % top-1.

Slide 9 — System architecture (60 s)

(Insert architecture diagram from docs/12-architecture.md.)

- React + Vite frontend on Vercel.
- Flask + gunicorn backend on Render free tier.
- PostgreSQL managed on Render — holds the curated 47-species catalog (47 care profiles + 94 toxicity records).
- PyTorch inference runs on the same backend process (no separate GPU node required for the academic scale).

Slide 10 — Documentation and engineering practices (45 s)

- Repository: 79+ files, 14-section English documentation set.
- Tests: unit, functional, integration (pytest + pytest-flask).
- CI: GitHub Actions runs lint + tests on every push.
- Reproducibility: train/val/test splits stored as CSV manifests with a fixed seed (42). Anyone can re-run and get the same numbers.
- Cleanup script: `scripts/cleanup.sh --hard` removes the entire workspace after grading.

Slide 11 — Future work (30 s)

- Expand catalog to 100+ species via curated PlantNet-300K subset.
- Export the model to CoreML for on-device inference (no server cost, no upload latency, full privacy).
- Plant disease overlay using PlantVillage.
- Push reminders for watering and fertilizing.

Slide 12 — Thanks + Q&A (15 s)

- Live demo URL: <https://leafLens.vercel.app> (replace after deploy)
- Code: <https://github.com/luigy23/leafLens>
- Questions?

Closing line: "If you've ever killed a plant by loving it too much, LeafLens is for you. Thank you."

Speaker notes — common questions to prepare for

1. **Why three architectures and not one?** Rubric requirement, but also good practice: a CNN and a transformer make different mistakes; comparing them gives a defensible model selection.
2. **Why not train from scratch?** ImageNet-pretrained backbones converge to greater than 90% in a single training day on this dataset; from scratch would have wasted the one-week budget for no measurable accuracy gain.
3. **How do you handle class imbalance?** Class-weighted cross-entropy loss with weights inversely proportional to class frequency, plus data augmentation (rotation, flip, color jitter). Confirmed by per-class F1 staying close to overall F1.
4. **What about pets — is the toxicity data reliable?** Curated from ASPCA's toxic plant database (the standard reference for US-based vets) cross-checked with the RHS plant finder. The UI explicitly states that this is informational, not a vet substitute.
5. **Why no IoT layer?** The instructor confirmed IoT was a suggestion, not a hard requirement for this specific project track. The deliverable focuses on the computer vision + software engineering scope.
6. **Why English documentation if you're a Spanish speaker?** The rubric requires English documentation and presentation. The codebase itself follows international conventions to make collaboration with future maintainers easier.
7. **How are uploaded images handled?** Decoded into memory, run through the model, discarded. No server-side persistence. Only the prediction outcome (species id, confidence, latency) is

stored — for analytics, not for retrieval.

16. Deployment Guide

End-to-end instructions to publish LeafLens to public URLs that any laptop or phone can reach.

Target platforms: - **Backend + database** → Render (free tier, Docker-based) - **Frontend** → Vercel (hobby tier, static)

Total cost: \$0. Total time once your accounts are set up: ~30 minutes.

0. Prerequisites

You need a free account on each of: - GitHub (probably already have one) - Render — sign up with GitHub at <https://render.com> - Vercel — sign up with GitHub at <https://vercel.com>

You also need the LeafLens repo pushed to GitHub. If you haven't done that:

```
cd ~/Documents/GitHub/leafLens
gh repo create leafLens --public --source=. --remote=origin --push
```

(Requires the `gh` CLI; install with `brew install gh` and `gh auth login` once.)

1. Backend — Render

The repository already contains `render.yaml` and `Dockerfile`. Render reads these automatically.

1.1 Add the model checkpoint to a release asset

The trained checkpoint (`models/checkpoints/best.pt`) is 90 MB — too large for git but small enough for a GitHub release.

```
# From the repo root, with the best.pt already chosen:
gh release create v1.0.0 models/checkpoints/best.pt models/checkpoints/class_names.json \
  --title "LeafLens model v1.0.0" \
  --notes "ResNet-50 / 92.38% test top-1 / 47 classes"
```

Note the URL of `best.pt` from the release page. Render will download it on container startup.

1.2 Update the Dockerfile to fetch the checkpoint

Edit `Dockerfile` to add a download step before `unicorn` starts:

```
# After installing dependencies, before CMD:
ARG MODEL_URL
RUN if [ -n "$MODEL_URL" ]; then \
    mkdir -p models/checkpoints && \
    curl -L -o models/checkpoints/best.pt "$MODEL_URL"; \
fi
```

Then in Render's web UI, set the Docker build arg `MODEL_URL` to the release asset URL from step 1.1.

1.3 Create the Render service

1. Log in to Render → **New +** → **Blueprint**.
2. Connect the `leafLens` GitHub repo.
3. Render reads `render.yaml`, creates: - `leafLens-api` (web service, Docker) - `leafLens-db` (PostgreSQL free)
4. Wait for the first build (~10 min — torch alone is heavy).
5. The service URL will be `https://leafLens-api.onrender.com` (or similar).

1.4 Seed the production database

The Postgres database is created empty. Run the seeder against it once:

```
# Locally, with the production DATABASE_URL exported:
export DATABASE_URL="$(render secrets show leafLens-db --connection-string)"
source .venv/bin/activate
python scripts/seed_db.py
unset DATABASE_URL
```

(If you don't have the `render` CLI, copy the connection string from the Render dashboard.)

1.5 Verify

```
curl https://leafLens-api.onrender.com/api/health
```

Should return `{"status": "ok", "num_classes": 47, ...}`.

2. Frontend — Vercel

2.1 Connect the repository

1. Log in to Vercel → **Add New** → **Project**.
2. Import the `leafLens` GitHub repo.
3. **Root directory**: set to `frontend`.

4. **Framework preset:** Vite (auto-detected).

5. **Environment variables:** add - `VITE_API_BASE_URL` = `https://leafLens-api.onrender.com`

2.2 Deploy

Click **Deploy**. First build is ~1 minute. The site goes live at a URL like `https://leafLens-<hash>.vercel.app`.

2.3 Set up a custom URL (optional)

In the Vercel project settings, you can rename to `leafLens.vercel.app` if available. Update the `ALLOWED_ORIGINS` env var on Render to match.

3. Smoke test the full deployment

```
# Health
curl https://leafLens-api.onrender.com/api/health

# Predict (with a sample image)
curl -X POST https://leafLens-api.onrender.com/api/predict \
  -F "image=@data/raw/house_plant_species/Pothos\ \ (Ivy\ arum\)/1.jpg"

# Open frontend
open https://leafLens.vercel.app
```

Then upload a phone photo through the web UI and confirm you see the species + care card.

4. Render free-tier notes

- The web service **sleeps after 15 minutes of inactivity**. First request after sleep takes 30–60 seconds while Render boots the container. Demo tip: hit `/api/health` once before walking on stage.
- PostgreSQL free instances **expire after 90 days**. For an academic project this is fine; for production, upgrade or migrate.
- Outbound bandwidth is metered (100 GB/month). LeafLens at academic scale is nowhere near that ceiling.

5. Rollback

If a deployment goes bad: - Render: dashboard → service → Deploys → click any previous deploy → **Redeploy**. - Vercel: dashboard → project → Deployments → click a previous one → **Promote to Production**.

Both providers keep a history of every deploy.

6. Tear-down (when the project is done)

- Render: Settings → Delete Service (both `leafLens-api` and `leafLens-db`).
- Vercel: Settings → General → Delete Project.
- GitHub: optional — `gh repo delete luigy23/leafLens --yes` to remove the repo.

Then run the local `scripts/cleanup.sh --hard` to free disk space on the laptop.

17. Presentation Cheat Sheet — Numbers, Paths, and Q&A Reference

Print this sheet (or keep it open on your phone) during the presentation. Everything a professor or peer might ask, in one page.

The headline numbers

Question	Answer
Best test top-1 accuracy	92.38% (ResNet-50)
Best test top-3 accuracy	98.06% (EfficientNet-B0)
Number of species	47
Number of training images	10,341
Number of validation images	2,216
Number of test images	2,217
Total dataset size	14,774 images, 4.85 GB
Number of architectures compared	3 (EfficientNet-B0, ResNet-50, ViT-Base)
Inference latency (warm, MPS)	~200 ms (ResNet-50)
API end-to-end latency target	< 3 s (NFR-01) — actual ~1.2 s cold

Local paths (your laptop)

Resource	Path
Repository root	<code>~/Documents/GitHub/leafLens</code>
GitHub remote	<code>https://github.com/luigy23/leafLens</code>
Python venv	<code>~/Documents/GitHub/leafLens/.venv</code>
Dataset (Kaggle cache)	<code>~/.cache/kagglehub/datasets/kacpergregorowicz/house-plant-species/versions/4/house_plant_species/</code>
Dataset symlink in repo	<code>data/raw/house_plant_species</code> → above
Train manifest	<code>data/processed/train.csv</code> (10,341 rows)
Validation manifest	<code>data/processed/val.csv</code> (2,216 rows)
Test manifest	<code>data/processed/test.csv</code> (2,217 rows)
Class distribution report	<code>data/processed/class_distribution.csv</code>
Curated species catalog	<code>backend/data/species.yaml</code> (47 entries)
Local SQLite database	<code>backend/leafLens.db</code> (gitignored)
Trained checkpoints	<code>models/checkpoints/{efficientnet,resnet50,vit}_best.pt</code>
Deployed checkpoint	<code>models/checkpoints/best.pt</code> (copy of resnet50_best.pt)
Evaluation reports	<code>models/checkpoints/*_best.eval.json</code>
Training histories	<code>models/checkpoints/*_history.json</code>
Backend log	<code>/tmp/leafLens_backend.log</code>
Frontend log	<code>/tmp/leafLens_frontend.log</code>
ViT training log	<code>/tmp/leafLens_train_vit.log</code>

Service	URL
Source code (public)	https://github.com/luigy23/leafLens
Backend (local dev)	http://localhost:5001/api/health
Frontend (local dev)	http://localhost:5173
Backend (production)	<i>set after Render deploy</i>
Frontend (production)	<i>set after Vercel deploy</i>
Kaggle dataset	https://www.kaggle.com/datasets/kacpergregorowicz/house-plant-species
ASPCA toxicity reference	https://www.asPCA.org/pet-care/animal-poison-control/toxic-and-non-toxic-plants
RHS plant finder	https://www.rhs.org.uk/plants/search-form

Architecture details (in case they grill you)

EfficientNet-B0

- **Source:** `timm.create_model('efficientnet_b0', pretrained=True)`
- **Params:** 4.1 M total, 60 K trainable in the frozen phase
- **ImageNet top-1:** 77.7%

ResNet-50 (deployed)

- **Source:** `torchvision.models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)`
- **Params:** 23.6 M total, 96 K trainable in the frozen phase
- **ImageNet top-1:** 80.9% (V2 weights)

ViT-Base/16

- **Source:** `timm.create_model('vit_base_patch16_224', pretrained=True)`
- **Params:** 85.8 M total, 36 K trainable in the frozen phase
- **ImageNet top-1:** 81.0%
- **Why it lost on our dataset:** 14k images / 47 classes is below ViT's data-hungry sweet spot. CNNs with strong inductive biases win in the small-data regime.

Training recipe (identical for all 3 architectures)

Parameter	Value
Optimizer	AdamW
Weight decay	1e-4
Learning rate (backbone)	1e-4 (after warm-up)
Learning rate (head)	1e-3
Backbone freeze epochs	3
Total epochs (cap)	15 (12 for ViT)
Early stopping patience	5 epochs
Loss function	Class-weighted cross-entropy
Image size	224 × 224
Batch size	32 (16 for ViT, memory-bound on MPS)
Augmentation	RandomResizedCrop, HorizontalFlip, Rotation($\pm 15^\circ$), ColorJitter(0.1)
Train/val/test split	70 / 15 / 15, stratified
Random seed	42
Device	Apple MPS (M4 Air, 16 GB)

Care card data sources

The catalog at `backend/data/species.yaml` was hand-curated from:

- **ASPCA Animal Poison Control Center** — toxicity data (the standard US veterinary reference)
- **Royal Horticultural Society (RHS) plant finder** — care requirements
- **Missouri Botanical Garden plant finder** — family, origin, descriptions
- **PI@ntNet** — scientific name disambiguation

Each of the 47 species has: - 1 care profile (watering, light, temp, humidity, fertilizer) - 2 toxicity records (cat + dog) - 34 species are pet-safe, 7 mildly toxic, 53 toxic out of 94 total records

? Likely questions and crisp answers

Q: Why three models? Rubric requires ≥ 3 for the “best practices” section. Also: a CNN and a transformer make different mistakes; comparing them validates the model choice.

Q: Why didn't ViT win? It's supposed to. ViT's advantage emerges with millions of images. With 14k images and 47 classes, ImageNet-pretrained CNNs have a stronger inductive bias and converge faster. Documented in Section 13.8.

Q: Why did you stop ViT early at epoch 3? Its trajectory was 3 percentage points behind ResNet-50 with shrinking gap-closing room. Continuing 9 more epochs would have cost ~2 GPU-hours of laptop time with no expected ranking change. Disclosed transparently in Section 13.3.

Q: How do you handle class imbalance? Two mechanisms: (1) class-weighted cross-entropy loss with weights inversely proportional to class frequency, and (2) data augmentation. Confirmed by per-class F1 staying close to weighted F1.

Q: How big is the model in production? The ResNet-50 checkpoint is 90 MB. Fits comfortably in Render's free-tier container image.

Q: What happens if I upload a photo of a dog or a car? The model still returns a top-1 prediction (it has no "unknown" class), but the confidence will be low (< 40%), which triggers the explicit "low confidence — try a better photo" UI banner instead of the result card.

Q: Is the toxicity info trustworthy? Sourced from ASPCA, the standard veterinary reference. The UI explicitly says it's informational, not a vet substitute. Section 5.2 lists this in non-functional requirements.

Q: Did you train from scratch? No — all three models use ImageNet pretrained weights and were fine-tuned with a frozen-backbone warm-up followed by differential learning rates. Standard transfer learning. Training from scratch would have cost weeks and lost accuracy.

Q: How is the dataset split deterministic? `scripts/split_data.py` uses `sklearn.train_test_split` with `random_state=42` and `stratify=labels`. The resulting CSV manifests are gitignored only because they reference local file paths — the seed and the script are version-controlled.

Q: Why no IoT? The course rubric lists IoT as worth 10% but the instructor confirmed it was a suggestion for the agriculture track. Our project track focuses on computer vision + software engineering scope.

Q: What's the deployment pipeline? Push to `main` → GitHub Actions runs lint + tests → Render auto-deploys backend (Docker) → Vercel auto-deploys frontend. Zero manual steps after the initial setup.

Q: Can the model handle multiple plants in one photo? No — it's a classifier, not a detector. Out of scope for this project; future-work item.

Q: Are uploaded photos stored? No. Section 7.3 / NFR-07. Only the prediction outcome (species id, confidence, latency) is logged for analytics.

Demo flow (for the live demo slide)

1. Open `http://localhost:5173` (or production URL)

2. Drop in **Pothos** photo → show 99% confidence, point at the toxic-to-pets badge
3. Drop in **Snake Plant** photo → show different care card (water every 14–28 days)
4. Drop in **out-of-distribution** photo (a flower, anything random) → show the low-confidence banner
5. Click **Catalog** → show the 47-species grid
6. Click a species → show the detail page

LeafLens — Folder Inventory

Everything for the AI final project lives inside this single folder:

```
~/Documents/GitHub/leafLens
```

Total size on disk: **~6.7 GB** (4.9 GB of that is the dataset).

This folder is **fully self-contained** — code, dataset, model checkpoints, documentation, slides, and the database file. You can move it to an external drive, hand it in, or zip it up without anything breaking.

Top-level map

Path	Size	What's in it
<code>README.md</code>	7 KB	Public-facing readme with badges and quickstart
<code>INVENTORY.md</code>	this file	What lives where
<code>CLAUDE.md</code>	3 KB	Engineering context (for future AI sessions)
<code>requirements.txt</code>	<1 KB	Pinned Python dependencies
<code>pytest.ini</code> , <code>Dockerfile</code> , <code>render.yaml</code> , <code>.env.example</code>	small	Config files
<code>.python-version</code>	<1 KB	Pins Python 3.13 for direnv / pyenv
<code>.github/workflows/ci.yaml</code>	small	GitHub Actions CI (lint + tests)
<code>.gitignore</code>	<1 KB	What git ignores
data/	4.9 GB	The dataset (raw images + split manifests)
models/	524 MB	PyTorch architecture code + trained checkpoints
backend/	172 KB	Flask API, ORM, curated species catalog YAML
frontend/	76 MB	React + Vite app (most of it is node_modules)
docs/	120 KB	17-section English documentation set
presentation/	8 MB	.pptx deck + HTML reveal.js deck + cheat sheet
scripts/	36 KB	All utility scripts
tests/	24 KB	Unit + integration tests
<code>notebooks/</code>	empty	For ad-hoc EDA, currently empty
<code>.venv/</code>	1.2 GB	Python virtualenv with torch + flask + etc.
<code>.dev-logs/</code>	small	Backend and frontend stdout when running dev.sh

Where each thing is

Dataset

```

data/
├── raw/
│   └── house_plant_species/          ← 14,774 images, 47 species folders
│       ├── African Violet (Saintpaulia ionantha)/
│       ├── Aloe Vera/
│       ├── Anthurium (Anthurium andraeanum)/
│       ├── ... (44 more)
│       └── ZZ Plant (Zamioculcas zamiifolia)/
└── processed/
    ├── train.csv                    ← 10,341 rows (70%)
    ├── val.csv                      ← 2,216 rows (15%)
    ├── test.csv                     ← 2,217 rows (15%)
    └── class_distribution.csv        ← per-class counts

```

Provenance. Kaggle dataset `kacpergregorowicz/house-plant-species` version 4. Originally downloaded via `scripts/download_dataset.py` to the kagglehub cache at `~/.cache/kagglehub/...` and then copied into this folder for portability.

AI / models

```

models/
├── architectures.py                ← EfficientNet, ResNet50, ViT factories
├── dataset.py                      ← PyTorch Dataset + transforms
├── train.py                        ← Training loop
├── evaluate.py                     ← Test-set evaluation
├── inference.py                    ← PlantClassifier used by the API
├── __init__.py
└── checkpoints/
    ├── best.pt                     ← Deployed model (= resnet50_best.pt)
    ├── efficientnet_best.pt        ← EfficientNet-B0 checkpoint (16 MB)
    ├── resnet50_best.pt            ← ResNet-50 checkpoint (90 MB)
    ├── vit_best.pt                 ← ViT-Base checkpoint (327 MB, partial training)
    ├── *_best.eval.json            ← Per-architecture metrics (top-1, top-3, F1, confusion ma
    └── *_history.json               ← Per-epoch training history

```

Backend

```

backend/
├── app.py                ← Flask application factory
├── wsgi.py               ← gunicorn entrypoint
├── config.py             ← Config classes (dev, test)
├── leaflens.db           ← SQLite (47 species + 47 care + 94 toxicity records)
├── data/
│   └── species.yaml      ← Curated knowledge base (all 47 species)
├── db/
│   ├── models.py         ← Species, CareProfile, ToxicityRecord, Prediction
│   ├── seed.py           ← Idempotent YAML → DB seeder
│   └── __init__.py
└── routes/
    ├── health.py         ← GET /api/health
    ├── predict.py        ← POST /api/predict
    └── species.py        ← GET /api/species, /api/species/<id>

```

Frontend

```

frontend/
├── package.json          ← React 18 + Vite + Tailwind
├── vite.config.js        ← Dev proxy to backend on port 5001
├── tailwind.config.js
├── postcss.config.js
├── index.html
├── public/leaf.svg       ← Favicon
├── src/
│   ├── main.jsx          ← Entry point
│   ├── App.jsx           ← Router
│   ├── index.css         ← Tailwind + custom CSS
│   ├── api/client.js     ← axios wrapper
│   ├── components/
│   │   ├── Header, Footer, UploadDropzone, ResultCard, CareCard, Lo
│   │   └── HomePage, CatalogPage, SpeciesDetailPage, AboutPage
│   └── pages/
└── node_modules/        ← Dependencies

```

Documentation (17 sections in English)

```
docs/
├── 01-introduction.md
├── 02-problem.md
├── 03-objectives.md
├── 04-state-of-the-art.md
├── 05-requirements.md
├── 06-use-cases.md
├── 07-data-model.md
├── 08-class-diagrams.md
├── 09-mockups.md
├── 10-api-catalog.md
├── 11-testing.md
├── 12-architecture.md
├── 13-results.md           ← Final metrics + ResNet-50 selection
├── 14-future-work.md
├── 15-presentation-outline.md ← Slide-by-slide script
├── 16-deployment-guide.md   ← Render + Vercel walkthrough
└── 17-presentation-cheatsheet.md ← Numbers + paths + Q&A for the defense
```

Presentation

```
presentation/
├── LeafLens.pptx           ← PowerPoint (12 slides, opens in Keynote/PowerPoint)
├── build_slides.js         ← Regenerate the .pptx
├── package.json
└── html/                  ← Reveal.js HTML deck (recommended)
    ├── index.html
    ├── styles.css
    └── README.md
```

Scripts

```
scripts/
├── setup.sh               ← One-shot install of venv + npm deps
├── dev.sh                 ← Start backend + frontend together
├── download_dataset.py    ← Kaggle → data/raw/ (not needed anymore – already copied)
├── split_data.py          ← 70/15/15 stratified split, seed 42
├── build_species_catalog.py ← Generate placeholder catalog entries
├── seed_db.py             ← Seed PostgreSQL/SQLite from species.yaml
└── cleanup.sh            ← Remove venv/datasets/checkpoints (--hard deletes the rep
```

Tests

```

tests/
├── conftest.py                ← Shared pytest fixtures + StubClassifier
├── unit/
│   ├── test_preprocessing.py
│   ├── test_split.py
│   └── integration/
│       ├── test_api_health.py
│       ├── test_api_predict.py
│       └── test_api_species.py

```

Running the project from this folder

```

cd ~/Documents/GitHub/leafLens

# 1. First time only: install dependencies
./scripts/setup.sh

# 2. (One-time) seed the database
source .venv/bin/activate && python scripts/seed_db.py

# 3. Launch backend + frontend together
./scripts/dev.sh

# Open http://localhost:5173 in the browser.
# Press Ctrl+C to stop everything.

```

Reproducing the training results

```

source .venv/bin/activate

# Splits are already in data/processed/ – no need to re-split.
# To retrain a model:
python -m models.train --arch resnet50 --epochs 15
python -m models.evaluate --arch resnet50

```

Quick checks (defense backup)

What	Where	Expected
Final test top-1 (ResNet-50)	<code>models/checkpoints/resnet50_best.eval.json</code> → <code>top1</code>	0.9238
Final test top-3 (ResNet-50)	same file → <code>top3</code>	0.9802
Number of species in DB	<code>sqlite3 backend/leafLens.db "SELECT COUNT(*) FROM species"</code>	47
Care profiles	<code>sqlite3 backend/leafLens.db "SELECT COUNT(*) FROM care_profiles"</code>	47
Toxicity records	<code>sqlite3 backend/leafLens.db "SELECT COUNT(*) FROM toxicity_records"</code>	94
Image count in raw dataset	<code>find data/raw -type f \(-iname '*.jpg' -o -iname '*.jpeg' -o -iname '*.png' -o -iname '*.webp' \) \ wc -l</code>	14774
Image count in train manifest	<code>tail -n +2 data/processed/train.csv \ wc -l</code>	10341

Cleanup (after grading)

```
./scripts/cleanup.sh          # Removes venv, models, datasets, node_modules
./scripts/cleanup.sh --hard   # Additionally deletes the entire repo
```

Public repository: <https://github.com/luigy23/leafLens>

The git remote contains everything in this folder *except* the large generated artifacts (`data/raw/` , `models/checkpoints/` , `.venv/` , `frontend/node_modules/` , `backend/leafLens.db` , `.dev-logs/`). All of those are in `.gitignore` because they're either huge or local-only.